ABSTRACT

The "Blur Detection using OpenCV and Flask" aims to develop a web-based tool that assesses image clarity using advanced blur detection algorithms. This platform facilitates seamless image uploads via a user-friendly interface. Leveraging OpenCV, the system applies sophisticated technique such as Laplacian variance method to analyze uploaded images in realtime. Users receive immediate feedback on the detected blur level, enhancing their ability to evaluate image quality effectively .In addition to providing blur detection feedback, the system includes a feature where images are automatically segregated based on their sharpness values. This segmentation process categorizes images into different groups (e.g., sharp, moderately sharp, and blurry) using predefined thresholds derived from the blur detection analysis. This system also incorporates data visualization through bar chart graphs. These graphs dynamically display the distribution of image sharpness across uploaded images, offering users a visual summary of the dataset's quality characteristics. This visual representation not only enhances user understanding but also facilitates data-driven decision-making in scenarios where image quality assessment is crucial. This system enhances workflow efficiency in fields like photography, medical imaging, and quality control, ultimately leading to better outcomes and user satisfaction.

TABLE OF CONTENTS

Title	Page No.
CHAPTER 1 INTRODUCTION	
1.1 Introduction to the Project	1
1.2 Deep Learning	1
1.2.1 Image Recognition	2
1.2.2 Convolutional Neural Networks	2
1.2.3 Supervised Learning	3
1.2.4 Back Propagation to Train Multilayer Architectures	3
1.3 Proposed Method	4
1.4 Literature Survey	4
CHAPTER 2 REQUIREMENT SPECIFICATION AND ANALYSIS	
2.1 Introduction	8
2.2 Functional Requirements	8
2.2.1 User Requirements	8
2.2.2 System Requirements	8
2.3 Non-Functional Requirements	9
2.3.1 Reliability	9
2.3.2 Performance	9
2.3.3 Usability	9
2.3.4 Scalability	9
2.3.5 Maintainability	9
2.4 Software Requirements	9
2.4.1 OpenCV	10
2.4.2 Tensor Flow	10
2.4.3 Pillow	10
2.4.4 Flask	10
2.4.5 Matplotlib	10
2.4.6 NumPy	10
2.5 Hardware Requirements	11

CHAPTER 3 SYSTEM DESIGN

3.1 Introduction 12

3.2 System Architecture Design 12

3.3 Flowchart 14

3.4 Use Case Diagram 15

3.5 Sequence Diagram 17

CHAPTER 4 SYSTEM IMPLEMENTATION

4.1 Modules 18

4.1.1 Dataset Collection 18

4.1.2 Data Pre-processing 18

4.1.3 Model Selection 18

4.1.4 Model Training and validation 19

4.1.5 Model Evaluation 19

4.1.6 Model Deployment 19

CHAPTER 5 RESULT AND ANALYSIS

20

CHAPTER 6 CONCLUSION

22

REFERENCES

23

LIST OF FIGURES

Figure No .	Title	Page No.
3.1	Architecture of the System	13
3.2	Architecture of the System of Face Recognition	14
3.3	Flowchart Diagram of the Project	15
3.4	Use-Case Diagram of the System	16
3.5	Use-Case Diagram of the User	16
3.6	Sequence Diagram of the System	17
5.1	Front End Page	20
5.2	Screenshot of Data	20
5.3	Screenshot of Training image	21
5.4	Face detection output	21

CHAPTER 1

INTRODUCTION

1.1 Introduction to the Project

Image blur is a common challenge in digital image processing, making the analysis and usability of visual data difficult. This issue is particularly critical in fields such as remote sensing, astronomy, microscopy, and medical imaging, where image clarity and precision are essential. Blur in images is typically caused by factors such as camera lens movement during capture, environmental interferences, and limitations in imaging devices. To tackle this problem, researchers are continually developing methods to detect and reduce image blur.

This project focuses on creating a reliable blur detection system using the Laplacian operator, with the support of OpenCV and Flask. The aim is to develop a tool that accurately identifies blurred and sharp images and provides a quantifiable sharpness value. This system will improve the analysis of image quality and increase the reliability of digital imaging applications across various domains. Computer vision, a crucial aspect of this project, involves enabling machines to interpret and understand visual data. Within computer vision, image processing techniques are used to analyse images and extract meaningful information.
Detecting blur is an essential component of image processing, and this project utilizes the Laplacian operator for this purpose. The Laplacian operator, a second-order derivative operator, highlights areas of rapid intensity change, making it effective for edge detection. By calculating the variance of the Laplacian of an image, we can measure its sharpness: a low variance suggests a blurred image, while a high variance indicates a sharp image.

Open-Source Computer Vision Library is a comprehensive library providing numerous tools for image processing, including functions to apply the Laplacian operator and calculate image sharpness. Flask, a lightweight web framework for Python, facilitates the development of a web-based interface for the blur detection system. To provide a more thorough analysis, this project includes graphs and visual representations of the sharpness value and the image itself. By integrating OpenCV and Flask, the project delivers a powerful solution for detecting blur, enhancing image quality analysis in various professional fields.

1.2 Image Processing

Image processing is an essential technology in computer science and engineering that involves the manipulation and analysis of digital images to improve their quality or extract valuable information. This field uses a variety of algorithms to perform tasks such as image enhancement, restoration, segmentation, and object recognition. The main goal is to transform images into a more desirable or informative format. Key concepts in digital image processing include pixel manipulation, colour space conversions, and image transformations. Common techniques like edge detection, filtering, and noise reduction are used to enhance image quality. Advanced methods, such as feature extraction and image segmentation, help identify and isolate specific elements within an image for further analysis.

Image processing has a wide range of applications. In healthcare, it assists in analysing medical images for diagnostics and treatment planning. In surveillance, it enhances video feeds for improved monitoring and security. In the automotive industry, it supports advanced driver-assistance systems (ADAS) for tasks like lane detection and obstacle recognition. Additionally, in multimedia, image processing enhances the quality of images and videos for entertainment and communication purposes. As digital images become increasingly prevalent, the importance of advanced image processing techniques continues to grow. This drives innovation and enhances capabilities across various industries, making image processing a critical area of research and application.

Image processing is extensively used in various industries, making significant contributions to different sectors. In the field of medical imaging, it is utilized to enhance and analyse images such as MRIs and CT scans, aiding in more accurate diagnoses and treatment planning. For remote sensing, image processing handles satellite or aerial images, which are vital for environmental monitoring, agricultural planning, and urban development. In surveillance, it improves video feed quality, thereby enhancing security and monitoring effectiveness. Within industrial automation, image processing is crucial for quality control and inspection, ensuring precision and efficiency in manufacturing processes. In the multimedia industry, it boosts the quality of images and videos, enriching the overall experience in entertainment and

communication. The broad application of image processing demonstrates its versatility and importance in advancing technology across numerous fields.

1.3 OpenCV

Open-Source Computer Vision Library is an open-source library developed by Intel in 1999 for computer vision and image processing. It offers a range of tools and algorithms for realtime image and video analysis and supports multiple programming languages, including C++, Python, Java, and MATLAB. The library excels in tasks such as filtering, geometric transformations, and colour adjustments, as well as advanced techniques like edge detection and noise reduction. It includes algorithms for object detection, like Haar cascades and feature detectors such as SIFT and SURF.

OpenCV also supports real-time video capture and processing, making it valuable for applications in surveillance and sports analytics. It integrates with machine learning frameworks for object classification and clustering. Additionally, it provides tools for 3D vision, including stereo vision and 3D reconstruction. With cross-platform support for Windows, Linux, macOS, Android, and iOS, OpenCV is widely applicable across various industries, including healthcare, automotive, security, and entertainment. Its open-source nature and active community make it a key resource for computer vision projects.

1.4 Flask

Flask is a versatile, open-source web framework for Python, designed with simplicity and flexibility in mind. Introduced in 2010 by Armin Ronacher, Flask provides a foundational structure for building web applications without enforcing a specific project layout or dependencies. This framework is built upon Werkzeug, a powerful WSGI library for handling HTTP requests, and Jinja2, a tinplating engine that simplifies HTML generation.

Flask's minimalistic design means it offers the essential tools needed for web development while allowing developers to integrate additional components as required. This makes it particularly suitable for small to medium-sized projects. Developers can extend Flask’s capabilities by adding libraries for features such as database interaction, user authentication, and form handling. Its adaptability, combined with comprehensive documentation and a

supportive community, has made Flask a popular choice for creating web applications. The framework's straightforward approach ensures that developers can build efficient and scalable applications while maintaining control over the components and structure of their projects.

1.5 Proposed Method

We propose an application that uses OpenCV's Laplacian algorithm for blur detection, integrated with Flask to offer a user-friendly web interface. This system processes images by calculating the variance of the Laplacian, a second-order derivative operator that detects edges and measures sharpness. The variance value provides a quantitative measure of sharpness, allowing the application to determine if an image is blurred or not blurred and to assign a separate sharpness value accordingly. The application provides real-time feedback on image clarity, distinguishing between blurred and sharp images with precision. By combining OpenCV's robust image processing capabilities with Flask's accessible web interface, users can easily upload images and receive immediate sharpness analysis.

1.5 Literature Survey

Raghav Bansal et al and title named by [1] refers that, it analyses edges and sharpness via the Laplacian operator to quantify blur severity. The approach includes Gaussian noise estimation for accurate variance calculation, adapting to image content. Utilizing OpenCV, the algorithm proves reliable and fast, validated through simulations across diverse images and noise levels. Comparative analyses affirm its advantages in image quality assessment, supported by a comprehensive literature survey.

Kanjar De and V. Masilamani and title named by [2] refers that, it aims to improve upon traditional methods by leveraging spectral characteristics to assess image quality more accurately. This approach is significant for its potential applications in diverse fields, including digital photography and medical imaging, where precise image assessment is crucial. The paper contributes to ongoing efforts in image processing research to develop advanced algorithms for evaluating and enhancing image clarity.

Roxanne A. Pagaduan et al and title by [3] refers that, it compares techniques based on accuracy, precision, recall, F-measure, and execution time using consistent Gaussian kernel and threshold values. Results show HWT excels in accuracy, FFT in precision, LAP in recall, and HWT in Fmeasure. MLAP

performs fastest. This study aids image processing practitioners in selecting optimal techniques for enhancing image quality before restoration.

Rupali Yashwant Landge and Rakesh Sharma and title named by [4] refers that this paper addresses the prevalent issue of blurry images in digital photography, caused by factors like exposure time and device handling. It emphasizes the need for efficient tools to detect and manage blurry images, potentially for automatic deletion to optimize storage and maintain image quality. The survey covers techniques ranging from transform-based methods using DCT or Wavelet transforms to those not requiring transformations, offering insights valuable for researchers and practitioners in digital image processing.

Hanghang Tong et al and title named by [5] refers that using Harr wavelet transform. Their method analyses edge types and sharpness to assess image quality, determining the presence and extent of blur. Experimental results validate the effectiveness of their approach in distinguishing blurred images from sharp ones.

Yutao Liu et al and title named by [6] refers that it addresses the lack of dedicated methods for objectively evaluating these common yet challenging image distortions. GPSQ leverages gradient magnitude and phase congruency to analyse local blurriness and integrates saliency detection to emphasize visually significant areas. By weighting the local structure map with the saliency map, GPSQ provides a comprehensive measure of visual quality. Experimental results demonstrate high consistency with subjective evaluations, validating its effectiveness in accurately assessing image quality in scenarios where clarity is crucial, such as photography and medical imaging.

Bolan Su et al and title named by [7] refers that it detects blur by analysing singular values at each pixel and classifies motion and defocus blur types using an alpha channel constraint, avoiding the need for image deblurring or kernel estimation. Experiments with 200 blurred and 200 sharp regions from 100 images show accurate detection and classification. The technique's applications include image segmentation, depth estimation, and information retrieval, showcasing its versatility in multimedia analysis tasks.

Boon Tatt Koik and Haidi Ibrahim and title named by [8] refers that it covers methods such as edge sharpness analysis, low depth of field image segmentation, blind image de-convolution, Bayes discriminant function, non-reference block analysis, directional high-frequency energy analysis for motion blur, and wavelet-based histogram with SVM. The survey highlights ongoing advancements and challenges in developing efficient algorithms capable of handling various types of blur in digital images.

Shanghang Zhang et al [9] refers that the framework that goes beyond traditional blur removal methods to automatically assess the desirability of image blur. Using a dilated fully convolutional neural network with pyramid pooling and boundary refinement layers, it estimates spatially-varying blur maps and

classifies blur into three desirability levels (good, neutral, bad). The approach integrates high-level semantic understanding and an attention mechanism to identify important image content. Training is conducted end-to-end with supervision from both pixel-wise blur responses and image-wise desirability levels. The authors also contribute a new large-scale dataset to overcome limitations in existing blur datasets. Their method achieves state-of-the-art performance on blur estimation and quality assessment tasks across multiple datasets.

Piotr Wozniak and Bogdan Kwolek and title named by [10] refers that, introduces an algorithm for visual place recognition that enhances traditional methods by integrating graph-based decisions on deep embeddings and blur detections. This approach addresses the lack of confidence estimation in recognition decisions and improves scene categorization accuracy, particularly in the presence of severe blur variations. Evaluations conducted on a large dataset, including images with significant blur and 6-DOF viewpoint changes captured by a humanoid robot, demonstrate the algorithm's effectiveness in recognizing relevant places.

Bhuvaneswari Polavarapu and Hema Mamidipaka and title named by [11] refers that introduces a method using Resnet-50 and edge detection techniques for classifying blurry images into types like defocus and motion blur. They address the challenge of blind image restoration by leveraging statistical features extracted from edge-detected images. The Resnet-50 model is chosen for its ability to handle gradient vanishing and optimize training via skip connections. The study utilizes the Kaggle blur dataset to train and evaluate their approach, demonstrating its efficacy in enhancing image quality through accurate blur classification.

Usman Ali and Muhammad Tariq Mahmood and title named by [12] refers that evaluates multiple blur measure operators for the challenging task of single image blur segmentation. It addresses the complexities of detecting and categorizing blur types like defocus and motion blur without prior information. Using a diverse dataset of real-world blurry images, the study compares the performance of these operators under a unified framework, emphasizing their effectiveness in accurately segmenting blurred and non-blurred regions. Results highlight variations in performance across different image imperfections, providing insights into the strengths and limitations of each operator in practical image restoration applications.

CHAPTER 2

REQUIREMENTSPECIFICATIONANDANALYSIS

2.1 Introduction

Requirement analysis is the process of creating a score for a system's effort. Requirement analysis is critical to the success or failure of a systems or software project. The requirements should be documented, actionable, measurable, testable, traceable, related to identified business need so opportunities, and defined to a level of detail sufficient for system design.

2.2 Functional Requirements

In software engineering and systems engineering, a functional requirement defines a function of a system or its component, where a function is described as a specification of behavior between outputs and inputs.

2.2.1 User Requirements

- User should be able to access the application in laptop using webcam.
- Users should be able to upload images from their local storage or capture images via the webcam for blur detection.
- User should be able to analyze uploaded image to determine if it is blurred or not blurred using OpenCV.
- Users should be able to access the system, which will calculate the sharpness value of the image using the Laplacian variance method and display.
- Users should be able to access the system, which will provide a graphical representation of the sharpness analysis.

2.2.2 System Requirements

- The system should be able to take image inputs from the user, allowing them to upload photos for analysis.
- The system should provide a user-friendly web application interface, enabling users to interact with the application easily and perform image uploads and view results.

- The system should analyze the uploaded image to determine if it is blurred or not blurred. It should use the Laplacian variance method to quantify the sharpness of the image.
- The system should display the sharpness value of the image, providing a numerical representation of how sharp or blurred the image is.
- The system should offer graphical representation of the image analysis, such as plotting the sharpness values to give a visual understanding of the image quality.

2.3 Non-Functional Requirements

In system engineering and requirements engineering, a non-functional requirement is a requirement that specifies criteria that can be used to judge the operation of a system, rather that specific behaviors.

2.3.1 Reliability

Ability to handle and recover unexpected error and conditions and regular backups should be implemented to prevent data loss.

2.3.2 Performance

The blur detection using OpenCV and Flask should utilize the Laplacian variance method to achieve at least 90% accuracy in determining image sharpness.

2.3.3 Usability

The user interface should be easy to navigate. The system should provide clear and concise instructions to users.

2.3.4 Scalability

The system should be able to handle an increasing number of users and data without degradation.

2.3.5 Maintainability

Proper documentation should be provided for all system components. The system should allow easy updates and maintenance.

2.4 Software Requirements

The project requires following software to run.

- Operating System – 64-bit Microsoft® Windows® 8/10 or MacOS 10
- Languages – Python, XML.
- IDE – Visual Studio Code
- Python Packages-TensorFlow, Kera's

2.4.1 OpenCV

OpenCV can be integrated with other libraries such as Tensor Flow, Kera's, and PyTorch for deep learning applications. OpenCV's robust algorithms allow for accurate detection and recognition of faces in images and videos.

2.4.2 Tensor Flow

Tensor Flow is a free and open-source software library for machine learning and artificial intelligence, it can be used across arrange of tasks but has a particular focus on training and inference of deep neural networks.

2.4.3 Pillow

Pillow is widely used for opening, manipulating, and saving different image file formats. It also supports various image filtering and enhancement techniques such as contouring, blurring and edge detection.

2.4.4 Flask

Flask is alight weight and web-friendly Python web framework, designed to make it simple to create web applications. It follows the Web Server Gateway Interface standard and allows

developers to quickly set up routes, handle HTTP requests, and render HTML templates. Flask's modular design encourages the use of extensions for additional functionality, making it customizable to suit variety of project requirements.

2.4.5 Matplotlib

Matplotlib is a comprehensive library for creating static, animated and interactive visualization in Python. It is used in data analysis to visualize trends, distributions and relationships within datasets.

2.4.6 NumPy

NumPy is a library for numerical computing in Python. It supports multi-dimensional arrays and matrices. It is widely used in scientific computing, data analysis, and machine learning.

2.5 Hardware Requirements

- RAM:8GB
- Storage: Minimum requirement of 256 GB SSD
- Camera: A high - definition webcam (1080p or higher).
- Intel Core Processor (i3 above)

.

CHAPTER 3

SYSTEM DESIGN

3.1 Introduction

System design is the process of defining the architecture, components, modules, interfaces and data for a system to satisfy specified requirements. In this process requirements are translated into a representation of software. Initially, the representation depicts a holistic view of software. Subsequent refinement leads to a design representation that is very close to source code. One could see it as the application of systems theory to product development. There is some overlap with the disciplines of system analysis, system architecture and system engineering.

3.2 System Architecture Design

System architecture is the conceptual model that defines the structure, behavior, and more views of a system. An architecture description is a formal description and representation of a system, organized in a way that supports reasoning about the structures and behaviors of the system.

The below steps involved in construction of the system architecture for the Figure 3.1.

- **Image**: Capture or upload an image for processing.
- **Blurred Image**: The system should use the Laplacian variance method to determine if an image is blurred by calculating its sharpness value and classifying it based on a predefined threshold.
- **Not Blurred Image:** The system should use the Laplacian variance method to determine if an image is not blurred by calculating its sharpness value and classifying it as clear based on a predefined threshold.
- **Sharpness Value**: The system should calculate the sharpness value of an image using the Laplacian variance method, providing a numerical representation of image clarity.
- **Identification**: The system should identify the image's clarity status by analyzing the calculated sharpness value and classifying it as either blurred or not blurred.

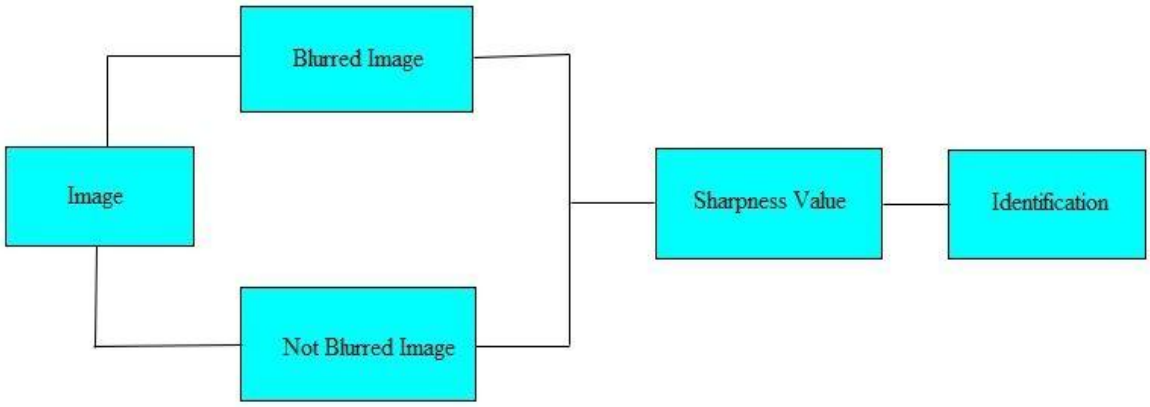


Figure 3.1: Architecture of the System of Blur detection

3.3 Flow Chart

A flowchart is one of the seven basic quality tools used in project management and it displays the actions that are necessary to meet the goals of a particular task in the most practical sequence. Also called as process maps, this type of tool displays a series of steps with branching possibilities that depict one or more inputs and transforms them to outputs. The advantage of flowcharts is that they show the activities in valve din a project including the decision points, parallel paths, branching loops as well as the overall sequence of processing through mapping the operational details within the horizontal value chain. Moreover, this particular tool is used in estimating and understanding the cost of quality for a particular process. This is done by using the branching logic of the workflow and estimating the expected monetary returns.

Figure 3.3 The blur detection system using OpenCV and Flask begins by initializing the system and loading necessary libraries. The user inputs an image by capturing it from a webcam or uploading a file. The system preprocesses the image and calculates the sharpness value using the Laplacian variance method. This sharpness value is compared to a threshold (e.g., 100). If below the threshold, the image is classified as blurred; if above, it is classified as not blurred. A line plot of the sharpness analysis is provided, and the sharpness value along with the classification result is displayed to the user, concluding the process.

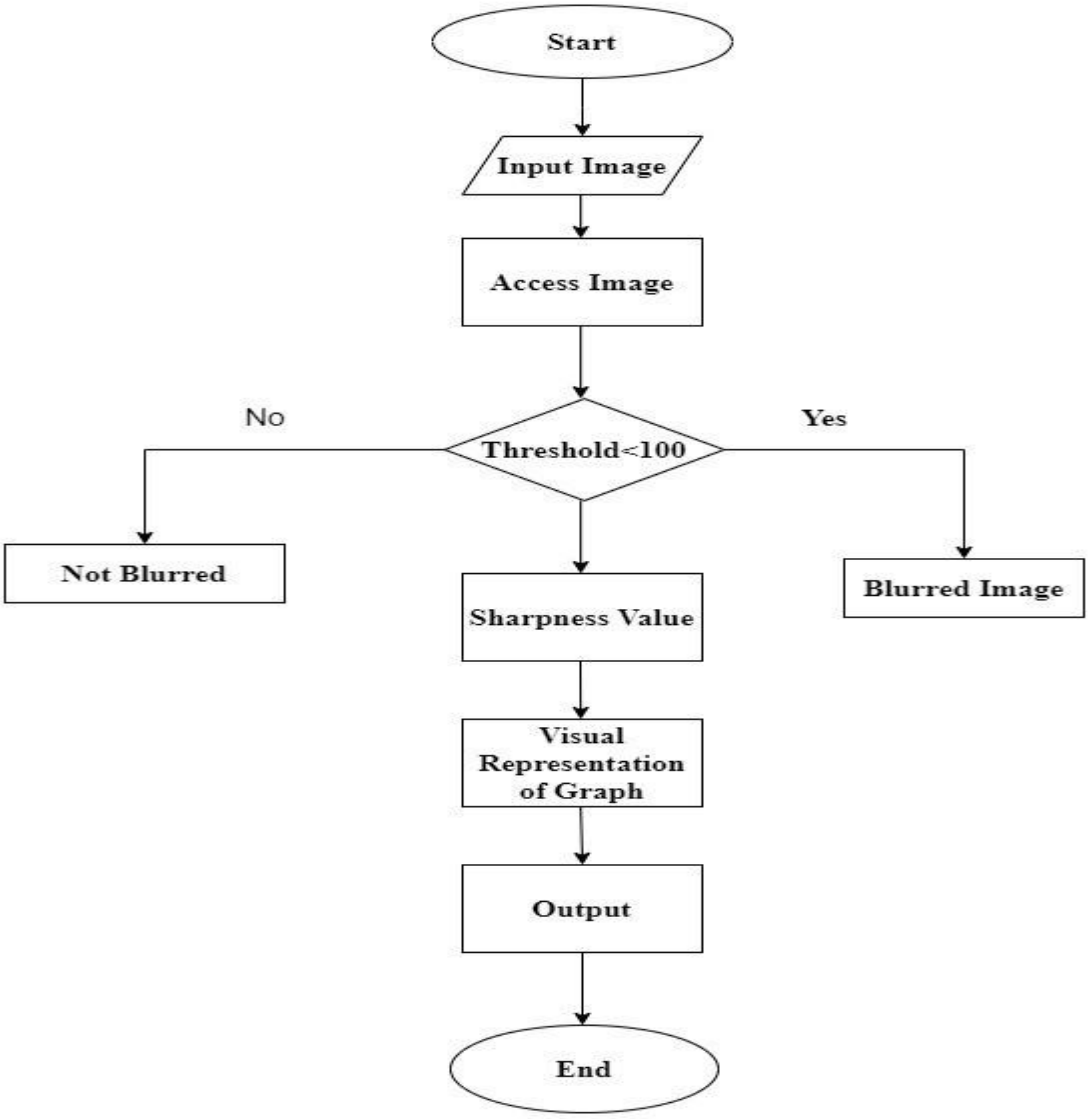


Figure 3.2: Flowchart Diagram of the Project

3.4 Use Case Diagram

A use case diagram in the Unified Modeling Languages is a type of behavioral diagram defined by and created format Use-case analys is. Its purpose is to present a graphical over view of the functionality provided by a system interims of factors, their goals, and any dependencies between those use cases. The main purpose of ause case diagram is to show what system functions are performed for which actor. Roles of the users in the system can be depicted.

Figure 3.3 shows the use-case diagram for the blur detection system using OpenCV and Flask, highlighting user interactions. The main actions include uploading an image for blur detection and analyzing the sharpness value. The user submits an image, and the system processes it for blur detection. The system calculates the sharpness value using the Laplacian variance method and classifies the image as either blurred or not blurred based on a predefined threshold. Additionally, the system provides a visual representation of the sharpness analysis through a graph, ensuring comprehensive feedback for the user.

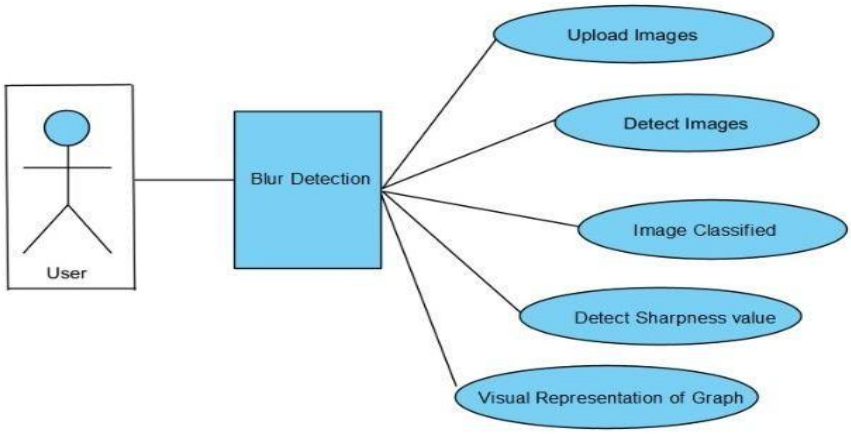


Figure 3.3:Use-case Diagram for the System

3.5 Sequence Diagram

A sequence diagram in Unified Modeling Language is a kind of interaction diagram that show processes operate with one another and in what order. It is a construct of a Message Sequence Chart. Sequence diagrams are sometimes called event diagrams, eventscenarios, and timing diagrams.

Figure 3.6 The sequence diagram for the blur detection system using OpenCV and Flask begins with the user selecting or capturing an image, which is then uploaded to the system. The raw image data undergoes preprocessing to ensure consistency and optimal processing, such as resizing and converting to gray scale. Next, the system applies the Laplacian variance method

to extract quality features and calculate the image's sharpness value. This sharpness value is then checked against a predefined threshold to determine if the image is blurred or not blurred. The system validates the sharpness value and the classification result, providing a graphical representation of the sharpness analysis. Finally, the system displays the sharpness value and the classification result to the user, ensuring a comprehensive and accurate analysis.

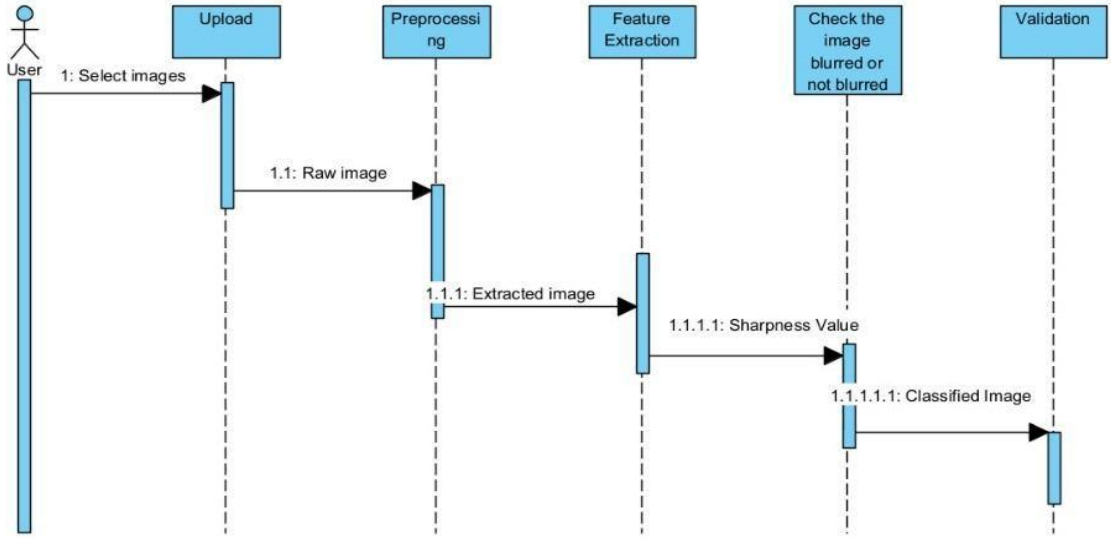


Figure 3.4: Sequence Diagram of the System

CHAPTER 4

SYSTEM IMPLEMENTION

4.1 MODULES

A module is a collection of source files and builds settings that let you divide your project into discrete units of functionality. A module can independently build, test, and debug each module.

The different modules are explained as below:

4.1.1 Dataset Collection

Dataset collection for blur detection involves compiling a diverse set of images that include both blurred and non-blurred photos. This dataset should represent various scenarios, such as different lighting conditions, motion blur, and varying degrees of sharpness. Sources for data can include public image repositories, private collections, and images captured using various cameras. Ensuring a balanced dataset with a wide range of examples enhances the model's robustness and accuracy in detecting blur.

4.1.2 Data Preprocessing

Data preprocessing in blur detection involves standardizing the collected images by resizing them to a consistent dimension, normalizing pixel values, Additional preprocessing steps may include cropping images to focus on regions of interest and applying filters to reduce noise. These steps optimize the dataset for analysis, improving the model's ability to accurately detect and measure image sharpness.

4.1.3 Model Selection

Model selection for blur detection involves choosing appropriate algorithms or techniques, such as the Laplacian variance method, to quantify image sharpness. This method is effective for measuring the degree of blur in an image by calculating the variance of the Laplacian, which

highlights edges. The selection of this model is based on its computational efficiency and accuracy in differentiating between blurred and non-blurred images.

4.1.4 Model Training and Validation

Model training and validation in blur detection involve feeding the pre-processed images into the selected algorithm, tuning parameters to optimize performance, and validating the results using techniques like cross-validation. This process ensures that the model accurately calculates sharpness values and can reliably classify images as blurred or not blurred. Iterative training and validation help in refining the model to enhance its generalization capability.

4.1.5 Model Evaluation

Model evaluation in blur detection assesses the performance of the trained model using metrics such as accuracy, precision, recall, and F1-score. The model is tested on a separate validation or test dataset to measure its effectiveness in detecting blurred images and calculating sharpness values. This evaluation ensures that the model meets predefined accuracy and reliability criteria before deployment.

4.1.6 Model Deployment

Model deployment in blur detection involves integrating the trained model into a web application using Flask, where it can process real-time image inputs from users. This includes optimizing the model for efficient execution, implementing interfaces for image upload and analysis, and ensuring robust performance across different operating conditions. The deployed system provides users with numerical sharpness values, graphical representations of sharpness analysis, and classification of images as blurred or not blurred, ensuring practical applicability in real-world scenarios.

CHAPTER 5 RESULT AND ANALYSIS

The below Figure 5.1 shows the Front end of the application, where the user can choose photos and extract image.

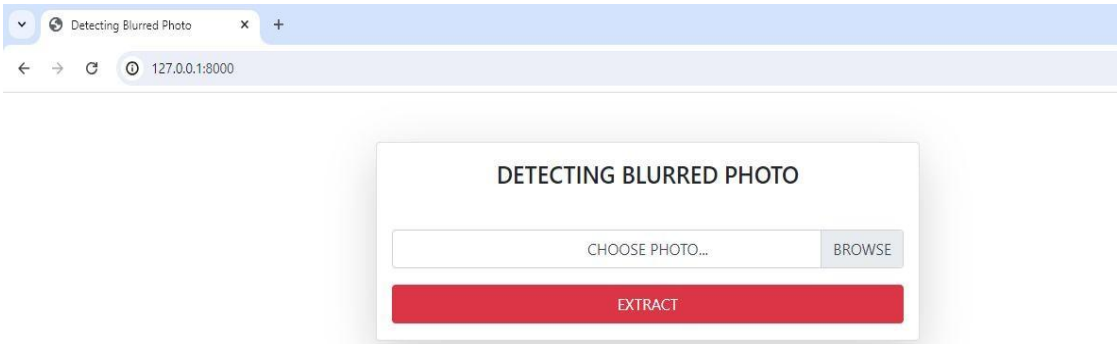


Figure 5.1: Front End Page

The Figure 5.2 shows the stored image. The images are used to detection blurred image or not blurred image.

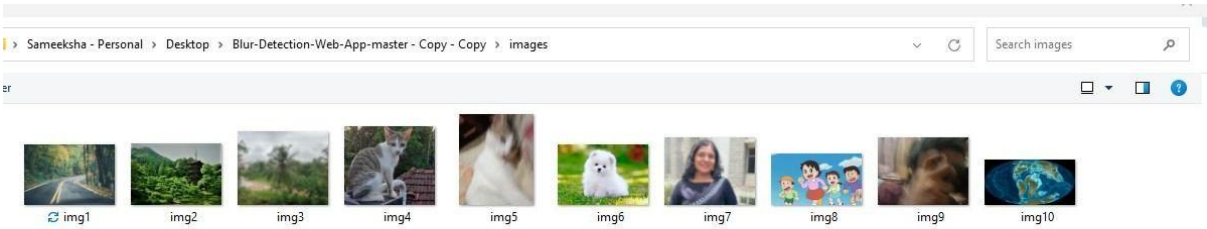


Figure5.2: Screenshot of Images

The Figure 5.3 shows the images which are not blurry.

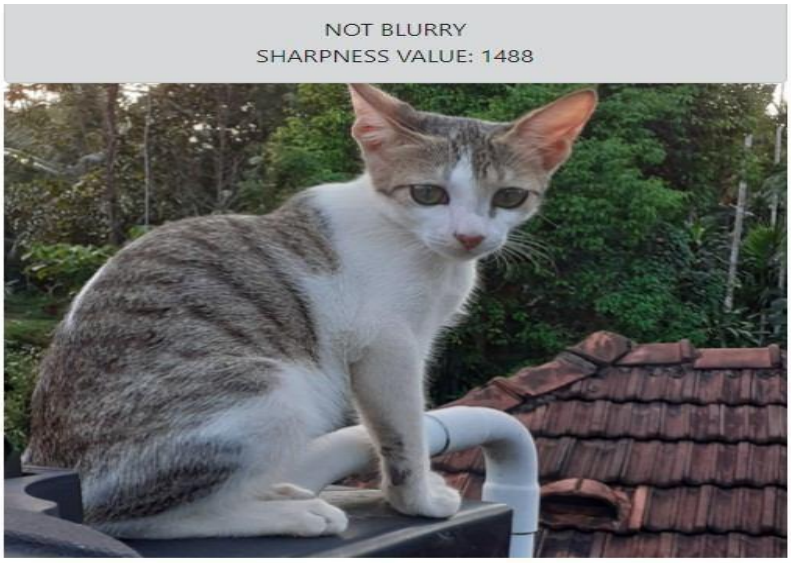


Figure 5.3: Screenshot of not blurred image

The Figure 5.4 shows the images which are blurry.

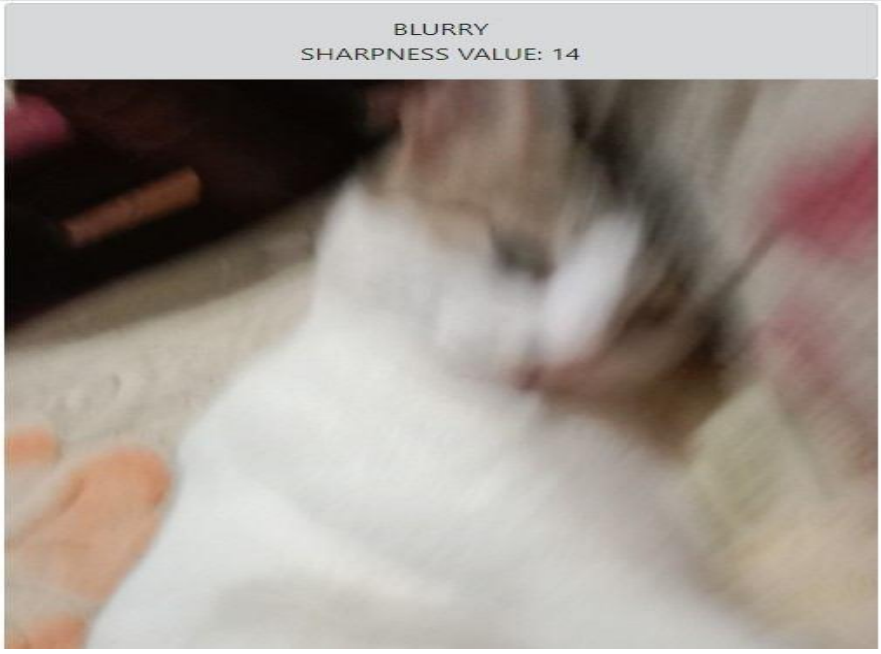
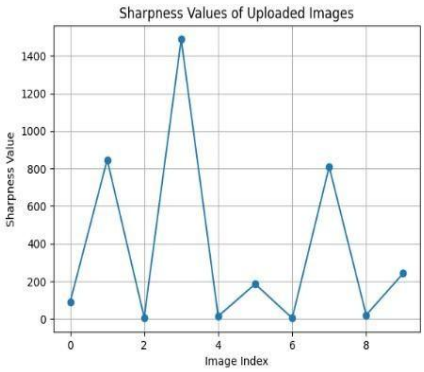


Figure 5.4: Screenshot of blurred image The Figure 5.5 shows the graphical representation of image and sharpness value.

DETECTING BLURRED PHOTO

SHARPNESS VALUES GRAPH



The Figure 5.5 shows the graphical representation of blurred and not blurred of images.

CHAPTER 6

CONCLUSION

Blur detection and analysis play a crucial role in ensuring high image quality for various applications. This project report outlines the implementation of blur detection using OpenCV's Laplacian algorithm, integrated with Flask for a web-based interface. The system efficiently classifies images as either blurred or not blurred by analysing the variance of the Laplacian, a second-order derivative operator that highlights intensity changes and edge details. By calculating this variance, the application provides a quantitative sharpness value, enabling precise evaluation of image clarity. The project successfully combines OpenCV's robust image processing capabilities with Flask's user-friendly web interface, facilitating easy image uploads and real-time analysis. In addition to classifying images, the system also generates graphs to visually represent the sharpness values, enhancing the interpretability of results. The integration of these tools ensures a seamless workflow for detecting blur and assessing image quality. The ongoing goal is to refine the system's accuracy and expand its functionality to handle a broader range of image types and conditions, thereby improving the reliability and usability of digital image analysis.